Practical 11

Part A:

Aim:

1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived.
Mark one field transient and show it is not saved.

Code:

Student.java

```
import java.io.Serializable;

public class Student implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;
    private String name;
    private double marks;

    // This field will not be serialized
    private transient String sessionToken;

    public Student(int id, String name, double marks,
                   String sessionToken) {
        this.id = id;
        this.name = name;
        this.marks = marks;
        this.sessionToken = sessionToken;
    }
```

```
    @Override
    public String toString() {
        return "Student{id=" + id
                + ", name='" + name
                + "', marks=" + marks
                + ", sessionToken=" + sessionToken + "}";
    }
}
```

SaveLoadDemo.java

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;

public class SaveLoadDemo {

    public static void main(String[] args) {

        String filename = "students.ser";

        Student[] students = {
            new Student(101, "Riya", 91.5, "TOKEN-A"),
            new Student(102, "Aman", 85.0, "TOKEN-B"),
            new Student(103, "Sara", 88.5, "TOKEN-C")
        };

        // Save the object array
        try (ObjectOutputStream out =
                     new ObjectOutputStream(
                         new FileOutputStream(filename))) {

            out.writeObject(students);
            System.out.println("Objects saved successfully.");
```

```
        } catch (IOException e) {
            System.out.println("Error saving objects: " +
e.getMessage());
            return;
        }

        // Read the object array into fresh objects
        try (ObjectInputStream in =
                    new ObjectInputStream(
                        new FileInputStream(filename))) {

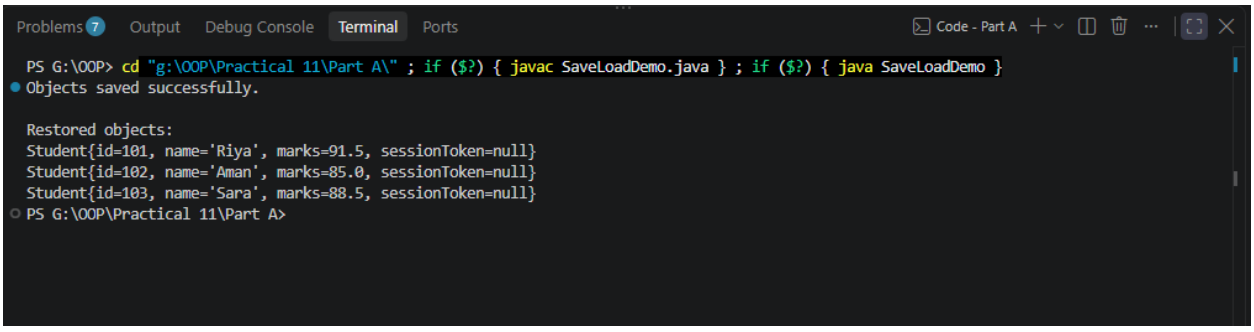
            Student[] restored = (Student[]) in.readObject();

            System.out.println("\nRestored objects:");

            for (Student student : restored) {
                System.out.println(student);
            }

        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error loading objects: " +
e.getMessage());
        }
    }
}
```

Output:



2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

Code:

LogAnalyzer.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;

public class LogAnalyzer {

    public static void main(String[] args) {

        Path directory = Paths.get("logs");
        String keyword = "error";

        try {
            // Create a directory for sample logs
            Files.createDirectories(directory);

            // Create three sample files
            Files.write(
                directory.resolve("app1.txt"),
                List.of(
                    "INFO Application started",
                    "ERROR Database connection failed",
                    "INFO Retry successful"
                ),
                StandardCharsets.UTF_8
            );
```

```
            Files.write(
                directory.resolve("app2.txt"),
                List.of(
                    "INFO User logged in",
                    "WARNING Low memory",
                    "ERROR File not found",
                    "INFO Application stopped"
                ),
                StandardCharsets.UTF_8
            );

            Files.write(
                directory.resolve("app3.txt"),
                List.of(
                    "INFO Server started",
                    "INFO Request processed"
                ),
                StandardCharsets.UTF_8
            );

        } catch (IOException e) {
            System.out.println("Could not create sample logs: "
                    + e.getMessage());
            return;
        }

        long totalLines = 0;
        long matchingLines = 0;

        try (var files = Files.list(directory)) {

            for (Path path : files.filter(Files::isRegularFile)
                                  .sorted()
                                  .toList()) {
```

```
                long fileLines = 0;
                long fileMatches = 0;

                try (BufferedReader reader =
                             Files.newBufferedReader(
                                 path, StandardCharsets.UTF_8))
{

                    String line;

                    while ((line = reader.readLine()) != null)
{
                        fileLines++;

                        if (line.toLowerCase()
                                .contains(keyword.toLowerCase()
)) {
                            fileMatches++;
                        }
                    }
                }

                long size = Files.size(path);

                totalLines += fileLines;
                matchingLines += fileMatches;

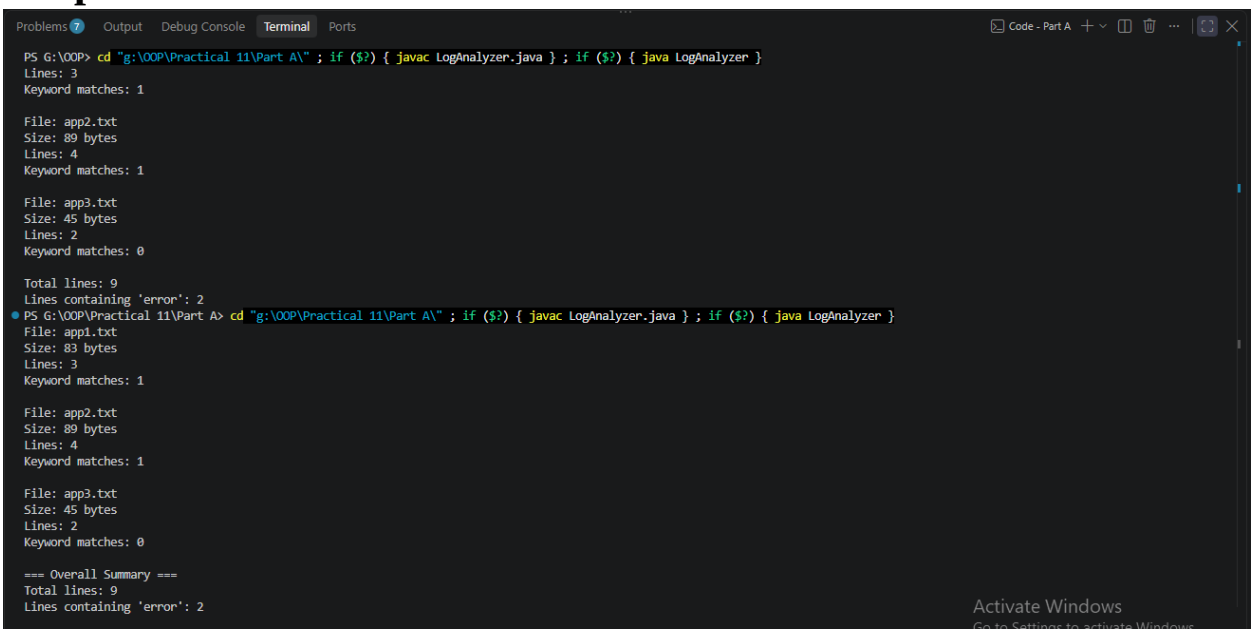
                System.out.println("File: " +
path.getFileName());
                System.out.println("Size: " + size + " bytes");
                System.out.println("Lines: " + fileLines);
                System.out.println("Keyword matches: " +
fileMatches);
                System.out.println();
```

```
            }

        } catch (IOException e) {
            System.out.println("Error analyzing logs: " +
e.getMessage());
            return;
        }

        System.out.println("=== Overall Summary ===");
        System.out.println("Total lines: " + totalLines);
        System.out.println("Lines containing '" + keyword
                + "': " + matchingLines);
    }
}
```

Output:



3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

Code:

FileTreeReport.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.FileVisitResult;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.SimpleFileVisitor;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

public class FileTreeReport {

    public static void main(String[] args) {

        Path root = Paths.get("sample-tree");
        Path report = Paths.get("file-report.txt");

        // Create sample directory structure
        try {
            Files.createDirectories(root.resolve("documents"));
            Files.createDirectories(root.resolve("images"));

            Files.writeString(
                root.resolve("readme.txt"),
                "This is a sample file.",
                StandardCharsets.UTF_8
```

```
            );

            Files.writeString(
                root.resolve("documents/notes.txt"),
                "Java NIO practice.",
                StandardCharsets.UTF_8
            );

            Files.writeString(
                root.resolve("images/info.txt"),
                "Sample image directory.",
                StandardCharsets.UTF_8
            );

        } catch (IOException e) {
            System.out.println("Could not create sample tree: "
                    + e.getMessage());
            return;
        }

        List<String> reportLines = new ArrayList<>();
        long[] totals = new long[2];
        // totals[0] = file count
        // totals[1] = total size in bytes

        reportLines.add("FILE TREE REPORT");
        reportLines.add("Root: " + root.toAbsolutePath());
        reportLines.add("");

        try {
            Files.walkFileTree(root, new
SimpleFileVisitor<Path>() {

                @Override
                public FileVisitResult visitFile(
```

```
                        Path file,
                        BasicFileAttributes attrs) {

                    if (attrs.isRegularFile()) {
                        String fileName =
file.getFileName().toString();
                        String relativePath =
root.relativize(file).toString();
                        long size = attrs.size();
                        Instant modified =
attrs.lastModifiedTime()
                                                .toInstant();

                        String entry = String.format(
                            "Name: %s | Path: %s | Size: %d
bytes"
                            + " | Modified: %s",
                            fileName,
                            relativePath,
                            size,
                            modified
                        );

                        reportLines.add(entry);

                        totals[0]++;
                        totals[1] += size;
                    }

                    return FileVisitResult.CONTINUE;
                }

                @Override
                public FileVisitResult visitFileFailed(
                        Path file,
```

```
                        IOException exc) {

                    reportLines.add(
                        "Could not access: " + file
                        + " | Reason: " + exc.getMessage()
                    );

                    return FileVisitResult.CONTINUE;
                }
            });

            reportLines.add("");
            reportLines.add("=== SUMMARY ===");
            reportLines.add("Total files: " + totals[0]);
            reportLines.add("Total size: " + totals[1] + "
bytes");

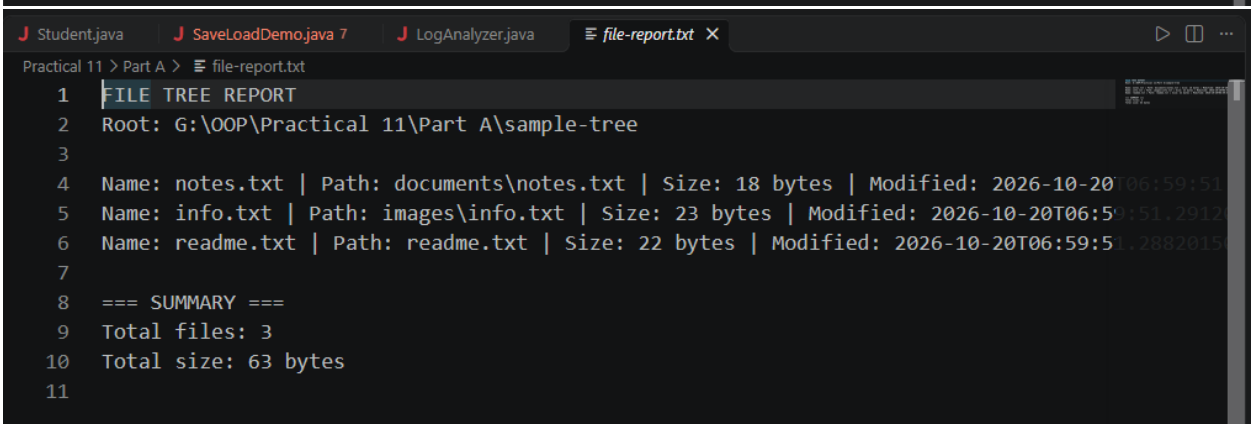
            Files.write(
                report,
                reportLines,
                StandardCharsets.UTF_8
            );

            System.out.println("Report generated: "
                    + report.toAbsolutePath());
            System.out.println("Total files: " + totals[0]);
            System.out.println("Total size: " + totals[1] + "
bytes");

        } catch (IOException e) {
            System.out.println("Error generating report: "
                    + e.getMessage());
        }
    }
}
```

Output:





Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What does it mean for a class to be Serializable, and why add serialVersionUID?

Ans.

A class is Serializable when it implements Java's Serializable interface, allowing its objects to be converted into a byte stream and saved or transferred for later reconstruction.

Example from Student.java:

```
import java.io.Serializable;

public class Student implements Serializable {

    private static final long serialVersionUID = 1L;
```

```
    private int id;
    private String name;
    private transient String sessionToken;
}
```

How it works:

1. ObjectOutputStream serializes the object's state into a file.
2. ObjectInputStream reads the serialized data.
3. Java reconstructs the object with its saved field values.

The Serializable interface is a marker interface: it declares no methods.

Why use serialVersionUID?

private static final long serialVersionUID = 1L;
It identifies the version of a serializable class. During deserialization, Java checks the stored identifier against the class's identifier. If they are incompatible, deserialization can fail with InvalidClassException.

What does transient do?

private transient String sessionToken;
It excludes the field from default serialization. After deserialization, the field receives its default value—in this case, null.

2. What is the difference between a byte stream and a character stream?

Ans.

Byte Stream	Character Stream
Processes raw bytes.	Processes text characters.
Commonly used for binary data, such as images and serialized objects.	Used for text files and character data.
Base classes include `InputStream` and `OutputStream`.	Base classes include `Reader` and `Writer`.
Examples: `FileInputStream`, `FileOutputStream`.	Examples: `FileReader`, `FileWriter`, `BufferedReader`.

3. How does NIO's Path/Files API differ from the old File class, and how is a directory stream iterated?

Ans.

`File` class	NIO `Path` / `Files` API
Represents a file or directory path.	`Path` represents a path; `Files` provides filesystem operations.
Provides basic methods such as `exists()`, `delete()`, and `mkdir()`.	Provides richer operations such as `readString()`, `copy()`, `move()`, and `walkFileTree()`.
Some operations return only a boolean, offering limited error details.	Many operations throw `IOException`, providing more informative error handling.
Directory listing commonly uses `list()` or `listFiles()`.	Directory iteration can use `Files.list()` and `DirectoryStream`.

A DirectoryStream<Path> lets you iterate over entries directly inside a directory.

```
import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

Path directory = Paths.get("logs");

try (DirectoryStream<Path> stream =
         Files.newDirectoryStream(directory)) {
```

```
    for (Path entry : stream) {
        System.out.println(entry.getFileName());
    }

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

The for loop visits each entry, and try-with-resources automatically closes the directory stream.